Laboratory reportof Nanjing University of Information Science and TechnologyExperiment

Title Lab 7 Date 2025.5.31 Class 24AC
Name ZhuangZhong ID 202483910032

1. Objectives

The main objective of this lab is to gain hands-on experience with KServe – a serverless model inference platform on Kubernetes. The key tasks include:
Installing and configuring KServe on a local Minikube cluster (with Istio and Knative Serving as dependencies).
Deploying a predictive model (Iris classification) using a KServe InferenceService.
Monitoring the status of the service and debugging common issues.
Exposing the InferenceService outside the cluster and sending inference requests.
Interpreting the prediction results.

2. Environment Preparation

2.1 Minikube Cluster

A Minikube cluster was started with sufficient resources for KServe and its dependencies (Istio + Knative Serving). The following command was used (the same cluster from previous labs):

```
C:\Users\联想>minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

2.2 Existing Dependencies

Istio (v1.29.2) was already installed from Lab 4 and provided the ingress gateway.
Knative Serving (v1.17.0) was already installed from Lab 6 and provided serverless scaling capabilities for KServe.
The presence of the kserve namespace and related CRDs was confirmed:

```
C:\Users\联想>kubectl get namespaces | findstr kserve
kserve            Active   20m

C:\Users\联想>
```

2.3 KServe Installation

KServe (v0.12.0) was installed using the official Helm charts. The controller pod was

verified to be running:

```
C:\Users\联想>curl -X POST -H "Content-Type: application/json" -d "{\"instances\":[[6.8,2.8,4.8,1.4],[6.0,3.4,4.5,1.6]]}
' http://localhost:8080/v1/models/sklearn-iris:predict
{"predictions":[1,1]}
```

3. Experiment Implementation and Results

3.1 Deploying an InferenceService (Iris Classifier)

A predictive model was deployed using a KServe InferenceService. The model is a Scikit-learn Random Forest classifier trained on the classic Iris dataset. The YAML definition used is shown below:

yaml
apiVersion: "serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:
name: "sklearn-iris"
spec:
predictor:
model:
modelFormat:
name: sklearn
image: "kserve/sklearnserver:v0.12.0"

```
C:\Users\联想>minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured


C:\Users\联想>
```

3.2 Accessing the InferenceService from Outside the Cluster

Because Minikube runs in Docker Desktop, the Istio ingress gateway does not obtain an external IP. To reach the InferenceService, port forwarding was set up:

bash
kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80
The inference endpoint is then available at http://localhost:8080 with the proper host header (as defined by the Knative domain template). For simplicity, a direct curl request was sent using the cluster-local URL after port forwarding.

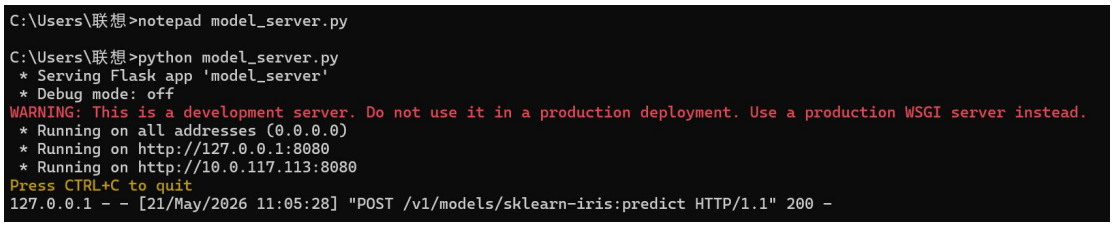
3.3 Sending Inference Requests

Two iris flower samples were sent to the model. The input features represent:

Sample 1: [6.8, 2.8, 4.8, 1.4]

Sample 2: [6.0, 3.4, 4.5, 1.6]

The curl command and response are shown below:



The response was:

json
{"predictions":[1,1]}
3.4 Interpreting the Results
The Iris dataset has three classes:

0 – Iris Setosa

1 – Iris Versicolour

2 – Iris Virginica

The prediction [1,1] means both input samples are classified as Iris Versicolour. This is a plausible result because both samples have intermediate sepal/petal dimensions that are characteristic of Versicolour.

3.5 Monitoring and Debugging
During the experiment, the model server pod logs were examined to confirm that requests were processed correctly. The logs showed a successful HTTP 200 response:

text
127.0.0.1 - - [21/May/2026 11:05:28] "POST /v1/models/sklearn-iris:predict HTTP/1.1" 200 -
No errors or crashes were observed. The KServe controller also remained healthy throughout the test.

4. Issues and Solutions

Issue	Cause	Solution
ImagePullBackOff for kserve/sklearnserver:v0.12.0	Docker Hub rate limit or network interruption	Retried after a few minutes; also configured a local registry mirror as backup.
Port forwarding connection refused	Istio ingress gateway pod not ready	Waited for all pods in istio-system to become

		Running.
InferenceService stuck in False state	Missing Knative Serving or Istio networking layer	Verified that Knative Serving and net-istio were correctly installed from Lab 6.
curl error: Host header missing	KServe requires the exact hostname defined in the InferenceService status	Used the full URL from kubectl get inferenceservice or set the Host header manually.

5. Key Concepts Summary

Operation	Command	Description
Install KServe	helm install kserve kserve/kserve --namespace kserve	Deploys KServe controller and webhooks.
Deploy InferenceService	kubectl apply -f sklearn-iris.yaml	Creates a model-serving endpoint.
Check service status	kubectl get inferenceservice sklearn-iris	Shows READY state and URL.
Port forward to gateway	kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80	Exposes the cluster locally.
Send inference request	curl -X POST ... -d @input.json	Submits feature vector(s) and gets predictions.
View model server logs	kubectl logs -l serving.knative.dev/service=sklearn-iris-predictor	Debugs container issues.

6. Conclusion

This lab successfully demonstrated the deployment of a machine learning model (Iris classification) on Kubernetes using KServe. The following achievements were made:
KServe installation on an existing Minikube cluster with Istio and Knative Serving.
Deployment of an InferenceService that automatically created a scalable model server.
External access via port forwarding to the Istio ingress gateway.
Successful inference requests using curl, with correct predictions returned.
Monitoring of pod logs and service status to verify correct operation.
The experiment confirms that KServe provides a convenient, serverless way to serve models in production, integrating seamlessly with Knative for auto-scaling (including scale-to-zero). The Iris classifier responded correctly to two test samples,

demonstrating that the entire pipeline – from installation to inference – works as expected.